Ce probleme pot fi rezolvate?

2024

Cuprins

1 Motivație **1**

2 Obiective **1**

3 Calculabilitate **2**
3.1 Noțiuni preliminarii - Tipuri de mulțimi 2
3.2 Demonstrație - Există probleme care nu pot fi rezolvate 3
3.3 Problema Opririi nu este calculabilă 4

4 Clasificarea problemelor **5**
4.1 Problemă recursivă 5
4.2 Problemă recursiv-enumerabilă 5
4.3 Probleme nerecursive 6

5 Teorema lui Rice **6**

1 Motivație

Intuitiv, știm că nu putem rezolva probleme care au o specificație ambiguă, pentru care nu avem suficiente informații sau care necesită un răspuns subiectiv. De asemenea, există probleme pentru care nu cunoaștem în acest moment răspunsul și nici nu știm dacă îl vom descoperi în viitor ("*Este conjectura lui Goldbach adevărată?*"). În acest curs însă vom discuta despre acele probleme care, matematic, nu pot fi rezolvate, indiferent de resursele pe care suntem dispuși să le investim.

Aceste probleme au numeroase aplicații practice în multe domenii, precum inteligența artificială ("*Cum putem ghida un robot să rezolve autonom o problemă?*") sau verificarea proprietăților unui program ("*Corespunde precis unei specificații formulate în limbaj natural*?" *"Este un virus"*?). Pentru a le aborda în practică va trebui să facem un compromis - să construim soluții care pot să nu funcționeze pentru orice intrare dar să acopere multe situații importante.

2 Obiective

În acest curs ne propunem să arătăm:

- Faptul că există probleme, precum Problema Opririi (Halting Problem) care nu sunt pot fi rezolvate pentru orice intrare.
- Cum putem clasifica problemele de decizie în funcție de abilitatea noastră de a le rezolva.
- De ce este dificil să demonstrăm că un program arbitrar are o anumită proprietate semantică (Teorema lui Rice).

3 Calculabilitate

3.1 Noțiuni preliminarii - Tipuri de mulțimi

Definiție 1. *O mulțime A este **numărabilă** dacă există o funcție bijectivă $index : \mathbb{N} \to A$ (putem numerota elementele mulțimii). Alternativ, spunem că o mulțime A este numărabilă dacă este finită sau are același cardinal cu $\mathbb{N}$.*

De exemplu, putem defini o regulă simplă pentru a numerota elementele mulțimii $\mathbb{Z}$:

n	1	2	3	4	5	...
index(n)	0	1	-1	2	-2	...

Follow-up 1:

Propuneți o regulă similară pentru a numerota elementele mulțimii $\mathbb{Q}$. Aveți grijă să nu asociați doi indici diferiți pentru același număr.

Teoremă 1. *Fie BIN, mulțimea tuturor secvențelor caracterelor binare infinite. BIN este **nenumărabilă**.*

Demonstrație. Plan: Presupunem prin absurd că o numerotare ar fi posibilă. Apoi, arătăm că există un element care nu poate face parte din numerotarea respectivă.

Pasul 1: Fie o numerotare arbitrară a mulțimii, $index : \mathbb{N} \to BIN$. Astfel, notăm cu $index(n)$ secvența cu indicele n din mulțime. Notăm cu $index(n)_i$ al i-lea caracter din secvența respectivă.

Pasul 2: Fie secvența binară, cu indicele x, definită conform următoarei reguli:

$$x_i = \begin{cases} 0 & index(i)_i = 1 \\ 1 & altfel \end{cases}$$

Putem vizualiza numerotarea obținută în următorul tabel:

n	$index(n)_1$	$index(n)_2$	$index(n)_3$	...	$index(n)_x$	...
1	**0**	1	1	...	...	...
2	1	**1**	0	...	...	...
3	1	0	**1**	...	...	...
...	...	...	...	...	...	...
x	**1**	**0**	**0**	...	**?**	...

Observăm că, prin construcția sa, secvența cu indicele x are cel puțin un caracter diferit față de toate secvențele din numerotare. Dar asta înseamnă că al x-lea caracter din secvența aleasă este diferit de propria sa valoare. □

Observație 1. *Această tehnică de demonstrație poartă în literatură numele de diagonalizare, folosită de George Cantor pentru a arăta că mulțimea $\mathbb{R}$ este nenumărabilă. În continuare, vom repeta acest tip de argumentație pentru a demonstra că există probleme care nu pot fi calculate.*

Follow-up 2:

Urmați aceiași pași pentru a demonstra că mulțimea numerelor reale din intervalul $[0, 1)$ este nenumărabilă.

Lema 1. *O submulțime infinită a unei mulțimi numărabile este numărabilă.*

Lema 2. *Fie A, o mulțime infinită **ne**numărabilă, respectiv B o mulțime numărabilă, astfel încât $x \in B$. $x \in A$. Atunci există elemente în A care nu sunt in B.*

Demonstrație. Dacă cele două mulțimi ar fi egale, A ar fi numărabilă (contradicție). □

Teoremă 2. *Fie Σ, un alfabet finit. Atunci mulțimea secvențelor de caractere (not. Σ^*) pe care le putem construi cu elementele din alfabet este numărabilă.*

Demonstrație. **Plan demonstrație**: Definim o relație de ordine $<_{lex}$ pentru orice pereche de elemente din Σ^*. Apoi arătăm cum putem genera efectiv elementele în funcție de ordinea impusă de relația de ordine.

Pasul 1: Deoarece Σ este finit putem defini o relație de ordine totală peste elementele sale (De exemplu, pentru alfabetul latin: $a < b < c < \dots$). Putem extinde această relație de ordine pentru secvențe de caractere astfel: Fie două secvențe finite de caractere $A = a_1a_2a_3\dots$, respectiv $B = b_1b_2b_3\dots$, astfel încât $A \neq B$. Dacă $|A| < |B|$ considerăm $A <_{lex} B$. Altfel, dacă $|A| = |B|$, fie k indexul primului caracter diferit. Atunci considerăm $A <_{lex} B \iff a_k < b_k$.

Pasul 2: Generăm iterativ toate secvențele de caractere de lungime 1, 2, 3, etc. Astfel, pentru fiecare din secvențele de lungime i, adăugăm pe rând câte un caracter din alfabet conform relației de ordine pentru a genera secvențele de lungime $i + 1$.

□

3.2 Demonstrație - Există probleme care nu pot fi rezolvate

Teoremă 3:

Mulțimea Mașinilor Turing este **numărabilă**.

Demonstrație. Putem codifica orice Mașină Turing (MT) ca o secvență de caractere, respectiv să validăm în timp finit că o secvență de caractere reprezintă o codificare validă a unei MT. Atunci, mulțimea tuturor MT este o submulțime a Σ^*. Asta înseamnă, conform Lemei 1, că mulțimea MT este numărabilă.
Pentru a numerota efectiv toate programele putem omite, din numerotarea inițiala peste $\Sigma*$, acele secvențe care nu reprezintă o codificare validă a unei MT. □

Observație 2. *În continuare, vom nota cu M_k MT cu indicele k din enumerarea tuturor MT.*

Teoremă 4:

Mulțimea problemelor de decizie este **nenumărabilă**.

Demonstrație. Fie $Hom(\mathbb{N}, Bool)$, mulțimea funcțiilor definite pe mulțimea $\mathbb{N}$ cu valori în $Bool$, respectiv mulțimea problemelor de decizie care primesc la intrare un singur număr natural.

Pasul 1: Presupunem prin absurd că există o numerotare a mulțimii $index : \mathbb{N} \to Hom(\mathbb{N}, Bool)$.

Pasul 2: Fie funcția $imp : \mathbb{N} \to Bool$:

$$imp(x) = not\ index_x(x)$$

Fie i indexul funcției imp din numerotare. Atunci, conform definiției, $imp(i) = not\ index_i(i) = not\ imp(i)$. Contradicție. □

Teoremă 5:

Există probleme care nu pot fi rezolvate.

Demonstrație. Mulțimea Mașinilor Turing este numărabilă, prin urmare mulțimea problemelor rezolvate folosind MT este și ea **numărabilă**. Totuși am demonstrat mai devreme că mulțimea tuturor problemelor de decizie este **nenumărabilă**. Atunci, conform Lemei 2, există probleme care nu au asociate o rezolvare. □

Există însă probleme practice care nu pot fi rezolvate?

3.3 Problema Opririi nu este calculabilă

Problema 1: Problema Opririi (Halting Problem)

Date de intrare:

- Mașina Turing M (un program)
- O secvență de caractere w (o intrare)

Problema de decizie: Se termină execuția lui M pe intrare w?

Teoremă 6:

Problema Opririi (PO) nu este calculabilă.

Demonstrație. **Plan demonstrație**: Replicăm tehnica de diagonalizare. Presupunem prin absurd că problema este calculabilă și folosim acest rezultat pentru a ajunge la o contradicție.

Pasul 1: Presupunem prin absurd că există o MT $Halt$ care verifică în timp finit dacă o MT M se termină pe intrarea w:

$$Halt(M, w) = \begin{cases} True & \text{dacă apelul } M(w) \text{ se termină} \\ False & \text{altfel} \end{cases}$$

Pasul 2: Fie pseudo-codul MT $ReverseHalt(i)$:

```
1: function ReverseHalt(i)
2:     if Halt(M_i, i) then
3:         while(True);
4:     else
5:         return
6:     end if
7: end function
```

$ReverseHalt$ accesează codul MT cu indicele i (M_i) din enumerarea tuturor MT și verifică rezultatul apelării lui $Halt$ pentru intrarea i.

- Dacă $Halt(M_i, i)$ se termină, atunci programul ciclează la infinit.
- Altfel, programul se termină imediat.

Pasul 3: Fie x indicele MT $ReverseHalt$. Ce se întâmplă când apelăm $ReverseHalt$ pentru propriul său indice?

- Dacă $Halt(M_x, x) == True$, atunci $ReverseHalt(x)$ ciclează la infinit. Contradicție.
- Dacă $Halt(M_x, x) == False$, atunci $ReverseHalt(x)$ se termină imediat. Contradicție

Putem vizualiza similaritatea acestui rezultat cu argumentul prin diagonalizare pe care l-am folosit anterior pentru a demonstra că o mulțime este nenumărabilă:

i	1	2	3	...	x	...
M_1	**Halt**	...	...	...	...	...
M_2	...	**Cycle**	...	...	...	...
M_3	...	...	**Halt**	...	...	...
...	...	...	...	...	...	...
M_x	**Cycle**	**Halt**	**Cycle**	...	**?**	...

□

4 Clasificarea problemelor

Fie $f_Q : \Sigma_Q \to \{True, False\}$, specificația problemei de decizie Q. Fie $A = \{x \in \Sigma_Q \mid f_Q(x) = True\}$, mulțimea de adevăr asociată.

4.1 Problemă recursivă

Definiție 2. *Funcția f_Q este **recursivă/decidabilă/calculabilă** dacă și numai dacă există un program P care calculează $f_Q(x)$, în timp finit, pentru **orice** element $x \in \Sigma_Q$. Dacă un astfel de program nu poate fi construit spunem că f_Q este **nerecursivă/nedecidabilă/necalculabilă**.*

Notăm cu R mulțimea problemelor recursive.

Exemplu 1: Probleme decidabile

- Este un număr natural x prim?
- Care este cel mai scurt drum de la A la B într-un graf orientat finit?
- Se termină MT M_k în cel mult t unități de timp?

Observație 3. *Dacă putem reformula Q ca o problemă de căutare într-un spațiu **finit** de posibilități, atunci Q este calculabilă. (ex: căutăm divizorii lui X în mulțimea numerelor mai mici decât X pentru a verifica dacă este prim).*

Strategii de demonstrație ($f_Q \in R$):

- Construim un program care corespunde specificației mulțimii A și care se termină în timp finit pentru orice intrare validă.
- Arătăm că putem reformula Q ca o instanță mai simplă a unei alte probleme cunoscute (Q_r) a cărei mulțime de adevăr este recursivă. ($Q \leq_T Q_r$)

4.2 Problemă recursiv-enumerabilă

Definiție 3. *Funcția f_Q este recursiv-enumerabilă/**semidecidabilă** dacă (alternativ):*

- *Există un program P care calculează răspunsul în timp **finit** pentru un element $x \in I_Q$ dacă $f_Q(x) = True$, dar poate să cicleze la infinit dacă $f_Q(x) = False$.*

$$P(x) = \begin{cases} True, & f_Q(x) = True \\ \perp, & f_Q(x) = False \end{cases}$$

- *Există un program G (generator) care la fiecare apel generează succesiv un element din A, astfel încât orice element $x \in A$ va fi returnat după un număr **finit** de apeluri ale lui G. Dacă A este finită, G poate să cicleze după generarea ultimului element.*

Observație 4. *Cele două definiții sunt echivalente. Putem folosi programul P pentru a construi generatorul G, respectiv putem folosi G pentru a implementa P.*

Exemplu 2: Probleme care sunt recursiv-enumerabile dar nu sunt recursive

- Există drum de la A la B într-un graf infinit?
- Se termină MT M_k pe intrarea w?
- Are o ecuație polinomială, cu coeficienți întregi, soluții în numere întregi? [1].

Strategii de demonstrație $f_Q \in RE \setminus R$:

- Conform definiției, putem fie să construim programul P, care se termină în timp finit pentru intrarea x dacă $f_Q(x) = 1$, fie generatorul echivalent care returnează toate elementele din A.
- Arătăm că putem reformula Q ca o instanță mai simplă a unei alte probleme cunoscute (Q_{rn}) a cărei mulțime de adevăr este RE. ($Q \leq_T Q_{rn}$)

Lema 3. *O problemă recursivă este și recursiv-enumerabilă. Reciproca nu este, în general, valabilă.*

Lema 4. *Problema complementară a unei probleme recursive (ex: Nu este numărul natural x un număr prim?) este tot recursivă.*

Observație 5. *Dacă f_Q este RE dar nu este R, atunci problema sa complementară nu este RE.*

Observație 6. *Principala dificultate asociată provine din faptul că nu putem explora un spațiu **infinit** de posibilități fără a avea o informație suplimentară care să ne permită să restrângem spațiul de căutare.*

4.3 Probleme nerecursive

Exemplu 3: Probleme care nu sunt nici RE

- Există drum de la A la B într-un graf infinit?
- Se termină MT M_k pe intrarea w?
- Are o ecuație polinomială, cu coeficienți întregi, soluții în numere întregi? [1].

Strategii de demonstrație $f_Q \notin R$

- Demonstrăm direct, prin reducere la absurd, că dacă mulțimea ar fi recursivă am obține o contradicție.
- Demonstrăm că putem reformula o problema cunoscută, Q_{nr} (ex: problema opririi), a cărei mulțime de adevăr este nerecursivă, ca o instanță mai simplă a problemei Q ($Q_{nr} \leq_T Q$).
- Analog, pentru a demonstra că o problemă nu este RE, arătăm că putem reformula o problemă cunoscută, Q_{nre} (ex: complementul problemei opririi), a cărei mulțime de adevăr nu este nici RE, ca o instanță mai simplă a problemei Q ($Q_{nre} \leq_T Q$).

5 Teorema lui Rice

Definiție 4. *Două programe P_1 și P_2 sunt echivalente (rezolvă aceeași problemă) dacă, pentru orice intrare w, $P_1(w) = P_2(w)$.*

Definiție 5. *Un program are o **proprietate extensională** Q dacă și numai dacă orice alt program echivalent cu el are aceeași proprietate.*

O problemă poate să admită multe tipuri de rezolvări. O proprietate extensională (semantică) a unei rezolvări se referă strict la relația dintre datele de intrare și cele de ieșire[1]. Cu alte cuvinte, o proprietate extensională este de fapt o proprietate a *problemei* rezolvate.[2].

Definiție 6. *O proprietate este **nebanală** dacă există cel puțin un program care are aceea proprietate, respectiv un program care nu are proprietatea respectivă.*

Teoremă 7: Teorema lui Rice

Orice proprietate **extensională nebanală** a unui program este nedecidabilă.

[1] De exemplu: Dacă rulăm $M(w)$, programul întoarce $2 * w$ pentru orice număr natural w?

[2] În contrast, o proprietate non-extensională (intensională) a unui program se referă la modul în care un program este implementat - de exemplu la structura sa, coding style, dimensiune, paradigma de programare, etc.

Observație 7. *Teorema ne spune că nu putem verifica automat dacă un program* ***oarecare*** *are o anumit proprietate semantică. Cu toate acestea, este posibil ca în multe cazuri particulare, relevante pentru noi, această verificare să fie posibilă.*

Demonstrație. Fie o proprietate extensională Q arbitrară, respectiv următoarea problemă:

$$CheckQ(M) - \text{Are Mașina Turing } M \text{ proprietatea } Q?$$

Fie T o MT care nu acceptă nicio intrare. Prin urmare, limbajul acceptat de T (respectiv mulțimea de adevăr a problemei rezolvate de T) este $\emptyset$. Presupunem, fără a pierde din generalitate, că T **nu** are proprietatea Q. În caz contrar, am aplica pașii din demonstrație pentru problema complementară $\overline{CheckQ}$ ("*Nu* are MT M proprietatea Q?") (v. Lema 4).

Plan demonstrație: Presupunem, prin reducere la absurd, că ar exista o MT M_Q care rezolvă problema $CheckQ$. Apoi arătăm că am putea folosi M_Q pentru a rezolva Problema Opririi, despre care știm deja că nu este calculabilă, pentru a ajunge la o contradicție. Concret, vom arăta acest lucru printr-o reducere (many-one) de la Problema Opririi (PO) la $CheckQ$: $PO(M, w) <_m CheckQ(M)$.

Pasul 1: Deoarece Q este o proprietate nebanală, știm că există cel puțin un program care are proprietatea respectivă. Fie $HasQ$ un astfel de program.

Pasul 2: Definim o funcție calculabilă t care transformă orice intrare validă a problemei PO într-o intrare particulară a problemei $CheckQ$.

```
1: t(M, w) =
2: function M_t(x)
3:     M(w);
4:     return HasQ(x);
5: end function
```

Pasul 3: Demonstrăm $PO(M, w) = True \implies CheckQ(M_t) = True$.

Dacă apelul inițial $M(w)$ se termină, atunci $M_t(x)$ este echivalent cu $HasQ$ deci programul $CheckQ(M_t)$ ar întoarce True.

Pasul 4: Demonstrăm $CheckQ(M_t) = True \implies PO(M, w) = True$.

Unica valoarea returnată de M_t este rezultatul execuției $HasQ(x)$. Pentru a putea executa $HasQ(x)$, subrutina $M(w)$ trebuie să se termine.

□

Follow-up 3:

Reproduceți pașii din demonstrația Teoremei lui Rice pentru a demonstra că următoarea proprietate nu este calculabilă:

$$CheckDouble(M) - \text{Pentru orice intrare } w \in \mathbb{Z}, \text{ întoarce Mașina Turing } M \text{ valoarea } 2 * w?$$

Follow-up 4:

Încercați aceeași abordare pentru a demonstra că următoarea proprietate ne-extensională nu este calculabilă:

$$CheckStates(M) - \text{Are Mașina Turing } M \text{ exact 4 stări?}$$

Ce observați?

Follow-up 5:

De ce a fost nevoie să tratăm special cazul în care proprietatea Q aparține programelor care nu acceptă nicio intrare? Analizați pasul 4.

References

[1] Martin Davis. "Hilbert's tenth problem is unsolvable". In: *The American Mathematical Monthly* 80.3 (1973), pp. 233–269.

[2] Neil D Jones. "Computability and complexity from a programming perspective". In: *Proof and System-Reliability* (2002), pp. 79–135.

[3] Michael Sipser. *Introduction to the Theory of Computation*. English. 3rd ed. Cengage Learning, 2012. ISBN: 978-1133187790.

[4] Alan Mathison Turing et al. "On computable numbers, with an application to the Entscheidungsproblem". In: *J. of Math* 58.345-363 (1936), p. 5.